

Cambio: se desea integrar otro servicio (<https://www.omdbapi.com/>) para incrementar los datos del detalle de una película. Obtener la lista de películas no cambia. Esto implica:

- En vez de "get movie by id", vamos a hacer "get movie by title". El id de la lista de películas actual hace referencia al id de TMDb, lo cual no tiene correlación en OMDB. A su vez, ambos servicios soportan la búsqueda de películas por título.
- Lógica de construcción del objeto movie resultante:
 - Si ambos servicios retornan resultados, se crea un objeto movie con propiedades combinadas:

```
private fun buildMovie( @Usage & elagarrigue
    tmdbMovie: Movie.MovieItem,
    omdbMovie: Movie.MovieItem
) =
    Movie.MovieItem(
        id = tmdbMovie.id,
        title = tmdbMovie.title,
        overview = "TMDB: ${tmdbMovie.overview}\n\nOMDB: ${omdbMovie.overview}",
        releaseDate = tmdbMovie.releaseDate,
        poster = tmdbMovie.poster,
        backdrop = tmdbMovie.backdrop,
        originalTitle = tmdbMovie.originalTitle,
        originalLanguage = tmdbMovie.originalLanguage,
        popularity = (tmdbMovie.popularity + omdbMovie.popularity) / 2.0,
        voteAverage = (tmdbMovie.voteAverage + omdbMovie.voteAverage) / 2.0
    )
```

- Si sólo uno de los servicios retorna un resultado, retornar ese resultado. Modificar el overview agregando el string "TMDB: " u "OMDB: " según corresponda.
- Si ninguno retorna un resultado, retornar vacío

Tareas (implementar en distintos commits):

- **Reemplazar TMDb get movie details by id por search movie by title.**
Include:

```
interface MoviesExternalSource {

    suspend fun getPopularMovies(): List<MovieItem>

    suspend fun getMovieById(id: Int): MovieItem
    suspend fun getMovieByTitle(title: String): MovieItem
}
```

```
@Serializable
data class RemoteMovie(
    val id: Int,
    val title: String,
    val overview: String,
    @SerializedName("release_date") val releaseDate: String,
    @SerializedName("poster_path") val posterPath: String,
    @SerializedName("release_date") val releaseDate: String?,
    @SerializedName("poster_path") val posterPath: String?,
    @SerializedName("backdrop_path") val backdropPath: String?,
    @SerializedName("original_title") val originalTitle: String,
    @SerializedName("original_language") val originalLanguage: String,
    val popularity: Double,
    @SerializedName("vote_average") val voteAverage: Double,
    val popularity: Double?,
    @SerializedName("vote_average") val voteAverage: Double?,
)
```

```

fun toDomainMovie() = Movie.MovieItem(
    id = id,
    title = title,
    overview = overview,
    releaseDate = releaseDate,
    releaseDate = releaseDate ?: "",
    poster = "${TMDB_IMAGE_BASE_URL}/w185$posterPath",
    backdrop = backdropPath.let { "${TMDB_IMAGE_BASE_URL}/w780$it" },
    originalTitle = originalTitle,
    originalLanguage = originalLanguage,
    popularity = popularity,
    voteAverage = voteAverage,
    popularity = popularity ?: 0.0,
    voteAverage = voteAverage ?: 0.0
)

```

```

internal class TMDBMoviesExternalSourceImpl(
    private val tmdbHttpClient: HttpClient,
) : MoviesExternalSource {

    override suspend fun getPopularMovies(): List<MovieItem> =
        getTMDBMovies().results.map { it.toDomainMovie() }

    override suspend fun getMovieById(id: Int): MovieItem =
        getTMDBMovieDetails(id).toDomainMovie()
    override suspend fun getMovieByTitle(title: String): MovieItem =
        getTMDBMovieDetails(title).apply { println(this) }.results.first().toDomainMovie()

    private suspend fun getTMDBMovies(): RemoteResult =
        tmdbHttpClient.get("/3/discover/movie?sort_by=popularity.desc").body()

    private suspend fun getTMDBMovieDetails(id: Int): RemoteMovie =
        tmdbHttpClient.get("/3/movie/$id").body()
    private suspend fun getTMDBMovieDetails(title: String): RemoteResult =
        tmdbHttpClient.get("/3/search/movie?query=$title").body()

    @Composable
    fun DetailScreen(viewModel: DetailViewModel, id: Int, onBack: () -> Unit) {
    fun DetailScreen(viewModel: DetailViewModel, title: String, onBack: () -> Unit) {
    }
    }

```

```

private fun NavGraphBuilder.homeDestination(navController: NavHostController) {
    composable(HOME) {
        HomeScreen(
            viewModel = getHomeViewModel(),
            onGoodMovieClick = {
                navController.navigate("$DETAIL/${it.id}")
                navController.navigate("$DETAIL/${it.title}")
            }
        )
    }
}

private fun NavGraphBuilder.detailDestination(navController: NavHostController) {
    composable(
        route = "$DETAIL/{MOVIE_ID}",
        arguments = listOf(navArgument(MOVIE_ID) { type = NavType.IntType })
    ) { backstackEntry ->
        val movieId = backstackEntry.arguments?.getInt(MOVIE_ID)
        val movieId = backstackEntry.arguments?.getString(MOVIE_TITLE)
        movieId?.let {

```

- Actualizar los tests
- Mover remote result la implementación de MoviesExternalSource a data/external/tmdb. Renombrar MoviesExternalSourceImpl a TMDBMoviesExternalSource.
- OMBD sólo va a ser utilizado para obtener el detalle de una película, no una lista de películas. Aparece el incumplimiento del principio IS en MoviesExternalSource, separar la interfaz en dos.

```

interface MoviesExternalSource {
    suspend fun getPopularMovies(): List<MovieItem>
}

interface MovieExternalSource {
    suspend fun getMovieByTitle(title: String): MovieItem
}

```

- Actualizar los tests
- Agregar el servicio OMBD. En este paso se puede reemplazar TMDb por OMBD para obtener la película por título.

```
private const val API_KEY = "d18da1b5da16397619c688b0263cd281"
private const val TMDb_API_KEY = "d18da1b5da16397619c688b0263cd281"
private const val OMBD_API_KEY = "a96e7f78"

private val ombdHttpClient =
    HttpClient {
        install(ContentNegotiation) {
            json(Json {
                ignoreUnknownKeys = true
            })
        }
        install(DefaultRequest) {
            url {
                protocol = URLProtocol.HTTPS
                host = "www.omdbapi.com"
                parameters.append("apikey", OMBD_API_KEY)
            }
        }
        install(HttpTimeout) {
            requestTimeoutMillis = 5000
        }
    }

@Serializable
data class RemoteMovie(
    @SerializedName(value = "Title") val title: String,
    @SerializedName(value = "Plot") val plot: String,
    @SerializedName(value = "Released") val released: String,
    @SerializedName(value = "Year") val year: String,
    @SerializedName(value = "Poster") val poster: String,
    @SerializedName(value = "Language") val language: String,
    @SerializedName(value = "Metascore") val metaScore: String,
    val imdbRating: Double,
) {
    fun toDomainMovie() = Movie.MovieItem(
        id = title.hashCode(),
        title = title,
        overview = plot,
        releaseDate = if (released.isNotEmpty() && released != "N/A") released else year,
        poster = poster,
        backdrop = poster,
        originalTitle = title,
        originalLanguage = language,
        popularity = imdbRating,
        voteAverage = if (metaScore.isNotEmpty() && metaScore != "N/A") metaScore.toDouble() else 0.0
    )
}
```

```

package edu.dyds.movies.data.external.omdb

import edu.dyds.movies.data.external.MovieExternalSource
import edu.dyds.movies.domain.entity.Movie
import io.ktor.client.*
import io.ktor.client.call.*
import io.ktor.client.request.*

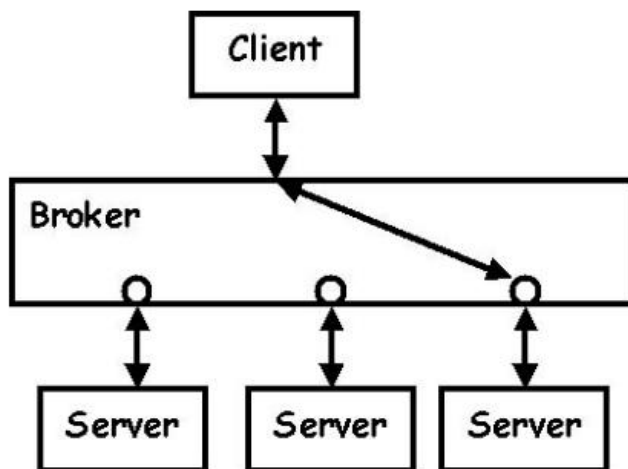
internal class OMDBMoviesExternalSource(
    private val omdbHttpClient: HttpClient,
) : MovieExternalSource {

    override suspend fun getMovieByTitle(title: String): Movie.MovieItem =
        getOMDBMovieDetails(title).toDomainMovie()

    private suspend fun getOMDBMovieDetails(title: String): RemoteMovie =
        omdbHttpClient.get(uriString = "http://www.omdbapi.com/?t=$title").body()
}

```

- Agregar un Broker que implemente MovieExternalSource que dependa de TMDBMoviesExternalSource y OMDBMoviesExternalSource que implemente la lógica del requerimiento.



- Implementar los tests para cubrir los casos del Broker.